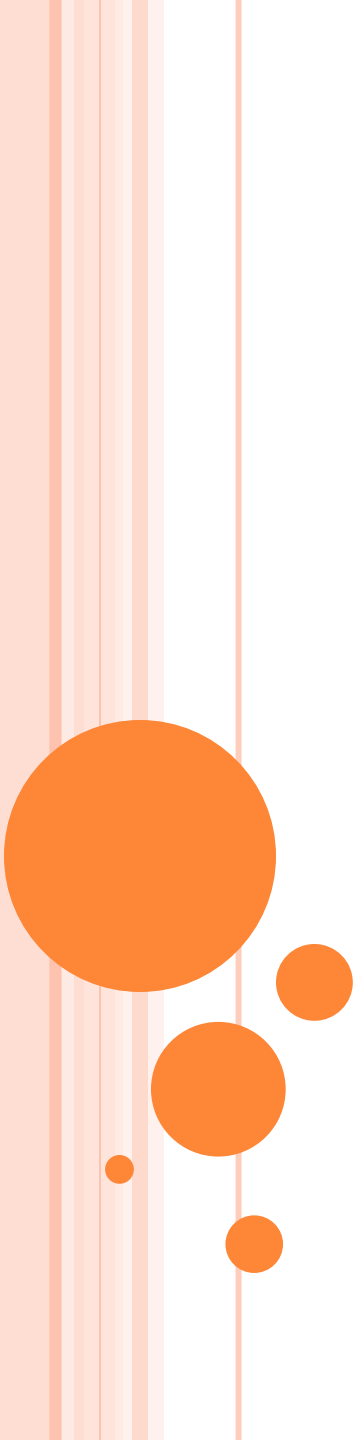




PROGRAMOWANIE GIER

WYKŁAD 2 – PAKIET PHYSICS, PLATFORMOWA GRA 2D, PRZENOSZENIE DANYCH MIĘDZY SCENAMI

Bogumiła Hnatkowska

KOMPONENTY Z PAKIETU PHYSICS

- Bryła sztywna (rigidbody)
- Zderzacz (collider)
- Materiały fizyczne (physics material)
- Efektor (effector)
- Złącze (joint)
- ...

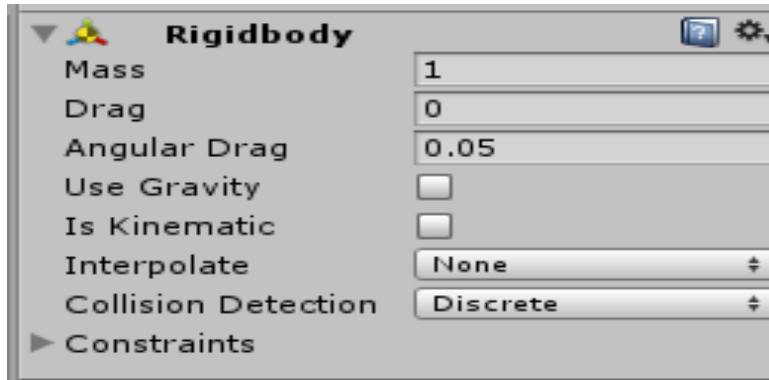


BRYŁA SZTYWNA

- Obiekt podlega grawitacji
- Gdy do obiektu są podłączone zderzacze, to dodatkowo może zmieniać kierunek w wyniku kolizji
- Nie powinien być przesuwany (modyfikacja komponentu Transform) programowo; do obiektu przykładamy siły, a silnik oblicza efekt ich działania
- Po zatrzymaniu jest „usypiany” (nie jest obsługiwany przez silnik aż do wybudzenia)
- Własność IsKinematic (kinematyczna bryła sztywna) – obiekt nie podlega siłom (np. grawitacji); jest sterowany przez skrypt lub animację, ale wykrywamy dla niego kolizje

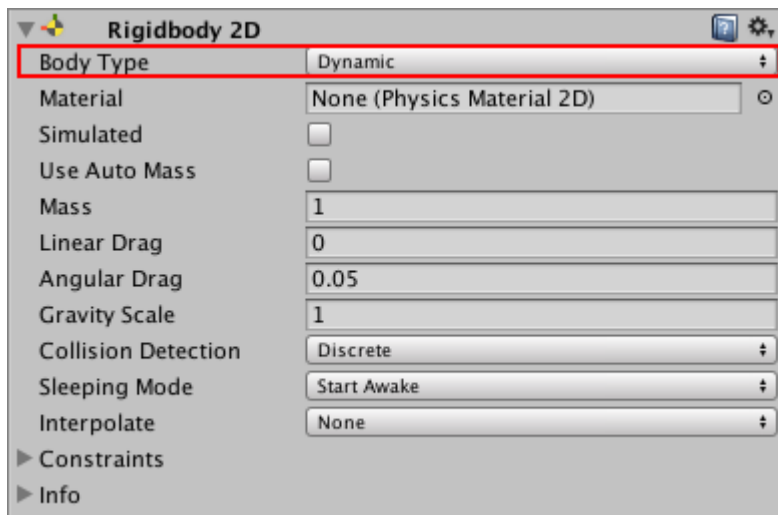


BRYŁA SZTYWNA, C.D.



Ważne własności:

- Mass
- Use Gravity
- IsKinematic



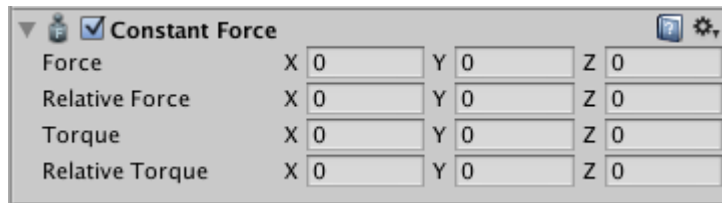
Rigidbody 2D:

- Mass
- Body Type: Dynamic, Kinematic, Static
- Gravity Scale

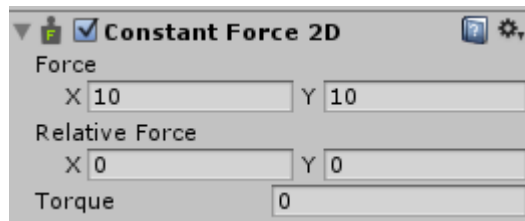


BRYŁA SZTYWNA, C.D.

- Do bryły sztywnej 2D/3D można dodać komponent reprezentujący stałą siłę (Constant force), przykładaną do ciała w ramach każdego Update, np. obsługa ruchu pocisku
 - ConstantForce



- ConstantForce2D



ZDERZACZ (COLLIDER)

- Byt, który reaguje na zderzenia
- Prymitywne typy zderzaczy 2D/3D: Circle, Box, Polygon, Capsule, Sphere
- Istnieją również bardziej zaawansowane, np. Edge, Mesh, czy Wheel
- Obiekt może mieć przypisanych wiele zderzaczy
- Zderzacz może mieć przypisany materiał fizyczny (physics material), np. śliski materiał, elastyczny materiał (dodatkowo parametryzowane)
- Zderzacze – w przypadku kolizji – generują zdarzenia, które mogą być przechwytywane przez skrypty: OnCollisionEnter (2D), OnCollisionStay (2D), OnCollisionExit (2D)
- Zderzacze mogą służyć do wykrywania faktu, iż jeden obiekt „wchodzi” w przestrzeń drugiego i wysyłać odpowiednie wiadomości (ustawiona opcja IsTrigger); taki obiekt nie stanowi przeszkody (inne przez niego przelatują): OnTriggerEnter (2D), OnTriggerStay (2D), OnTriggerExit (2D)

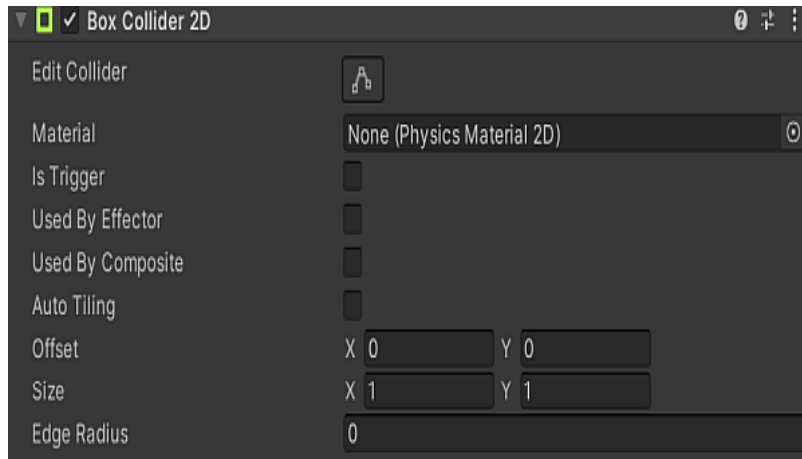


ZDERZACZ (COLLIDER), C.D.

- Typy zderzaczy (w zależności od kontekstu użycia):
 - Statyczny zderzacz (Static collider) – np. dodany do obiektu bez komponentu Rigidbody (przeszkody, których się nie przesuwa) lub do obiektu z Rigidbody oznaczonym jako static
 - Zderzacz dynamiczny (Dynamic collider) – dodany do obiektu z komponentem Rigidbody:
 - Zderzacz bryły sztywnej (Rigidbody collider) – IsKinematic=false; reaguje na kolizje (np. odbija się); może się zderzać z innymi zderzaczami (w tym statycznymi), co wpływa na jego położenie
 - Zderzacz kinematycznej bryły sztywnego (Kinematic Rigidbody collider) – IsKinematic=true; nie reaguje na kolizje i siły, ale inne ciała mogą się od niego odbijać; może być przesuwany w ramach skryptów



ZDERZACZ (COLLIDER), C.D.



- Ważne właściwości:
 - Edit Collider – pozwala dopasować kształt zderzacza do obiektu
 - Material
 - IsTrigger
 - Used by effector (Tylko dla 2D)



ZDERZACZ (COLLIDER), C.D.

- Obsługa zdarzenia OnCollisionEnter/ OnCollisionStay/OnCollisionExit
- Uwaga: wzbudzane domyślnie jedynie, gdy jeden ze zderzaczy ma powiązany komponent Rigidbody (nie kinematyczne, chyba że zmienimy ustawienia edytora)
- Parametrem jest obiekt kolizji (Collision): przechowuje m. in. punkty kontaktu i prędkość

```
void OnCollisionEnter(Collision collision) {  
    foreach (ContactPoint contact in collision.contacts) {  
        print(contact.thisCollider.name + " hit " + contact.otherCollider.name +  
            " with velocity: " + collision.relativeVelocity);  
        print("Points colliding: " + collision.contacts.Length);  
  
        Debug.DrawRay(contact.point, contact.normal, Color.white, 5.0f, false);  
    }  
}
```



ZDERZACZ (COLLIDER), C.D.

- Obsługa zdarzenia
OnTriggerEnter/OnTriggerStay/
OnTriggerExit
- Parametrem jest obiekt zderzacza, ten ma m. in. referencję do obiektu gry, do którego jest podłączony

```
public class ExampleClass : MonoBehaviour {  
    void OnTriggerEnter(Collider other) {  
        Destroy(other.gameObject);  
    }  
}
```



ZDERZACZ (COLLIDER), C.D.

Collision detection occurs and messages are sent upon collision

	Static Collider	Rigidbody Collider	Kinematic Rigidbody Collider	Static Trigger Collider	Rigidbody Trigger Collider	Kinematic Rigidbody Trigger Collider
Static Collider		Y				
Rigidbody Collider	Y	Y	Y			
Kinematic Rigidbody Collider		Y				
Static Trigger Collider						
Rigidbody Trigger Collider						
Kinematic Rigidbody Trigger Collider						

- Można obsłużyć kolizje dwóch kinematycznych/statycznych obiektów na scenie przez ustawienie:

Edit > Project Settings > Physics > Contacts Pair Mode" to "Enable All Contacts

Contact Pairs Mode
Broadphase Type
World Bounds

Center
Extent

Default Contact Pairs

- ☒ Default Contact Pairs
- Enable Kinematic Kinematic Pairs
- Enable Kinematic Static Pairs
- Enable All Contact Pairs

ZDERZACZ (COLLIDER), C.D.

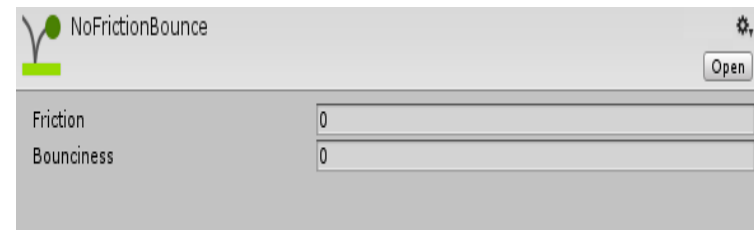
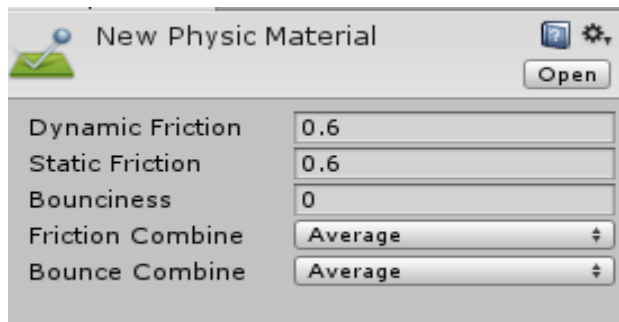
Trigger messages are sent upon collision

	Static Collider	Rigidbody Collider	Kinematic Rigidbody Collider	Static Trigger Collider	Rigidbody Trigger Collider	Kinematic Rigidbody Trigger Collider
Static Collider					Y	Y
Rigidbody Collider				Y	Y	Y
Kinematic Rigidbody Collider				Y	Y	Y
Static Trigger Collider		Y	Y		Y	Y
Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y
Kinematic Rigidbody Trigger Collider	Y	Y	Y	Y	Y	Y



MATERIAŁ FIZYCZNY

- Element, który pozwala sterować siłą tarcia (friction) i odbić (sprężystość) (bounciness) między zderzającymi się obiektami
- Utworzenie: Assets > Create > 2D > Physics2D Material



- Materiał fizyczny wystarczy przeciągnąć na obiekt ze zderzaczem



ZŁĄCZA (JOINTS)

- Bryłę sztywną można połączyć z inną lub do ustalonego punktu w przestrzeni z wykorzystaniem złącz
- Przykładowe złącza 2D:
 - Fixed Joint 2D – łączy dwa obiekty, które mają się zachowywać jakby były na sztywno połączone
 - Hinge Joint 2D – łączy dwa obiekty, ograniczając ich ruch, jakby były połączone zawiasem lub łączy obiekt z punktem pozwalając na jego rotację wokół ustalonej osi
 - Spring Joint 2D – obiekty połączone sprężyną
- Przykładowe złącza 3D:
 - Fixed Joint
 - Hinge Joint
 - Spring Joint
 - Character Joint



EFEKTOR (EFFEKTOR) – GRY 2D

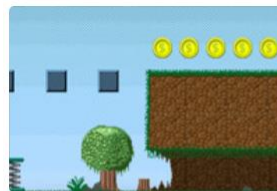
- Efektor – komponent, który pozwala do zderzaczy przyłożyć siły lub zmienić ich własności
- Przykładowe efektory 2D:
 - PlatformEffector2D – stosowany do platform, np. w celu umożliwienia postaci przechodzenia przez platformę w jednym kierunku
 - PointEffector2D – pozwala przyłożyć siłę punktowo, np. efekt wybuchu lub zasysania (czarna dziura)
 - SurfaceEffector2D – pozwala uzyskać efekt pasa transmisyjnego
 - AreaEffector2D – pozwala przyłożyć siłę w obszarze zdefiniowanym przez połączony zderzacz w trakcie kontaktu (np. wchodzenie po pionowej ścianie)



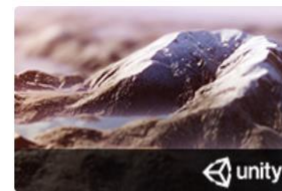
EFEKTORY – PRZYKŁAD ZASTOSOWANIA

- Demonstracja efektorów: powierzchni, platformy, punktowego dla gry 2D zbudowanej z wykorzystaniem zasobów ze sklepu
- 3 platformy zbudowane jako sześciiany (obiekty 3D):
 - Druga – efektor platformy (przenikanie w jedną stronę)
 - Trzecia – efektor powierzchni (taśma)
- Efektor punktowy zostanie wykorzystany do rozrzucenia monet po zderzeniu z wrogiem

- Kamera podąża za postacią



GAMERTOSE
Pixel Platformer Pack
★★★★★ (4)
FREE



UNITY TECHNOLOGIES
Terrain Tools Sample Asset ...
★★★★★ (39)
FREE

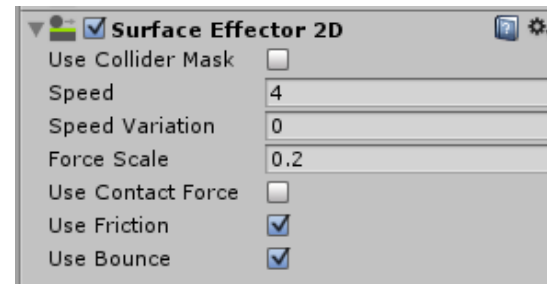
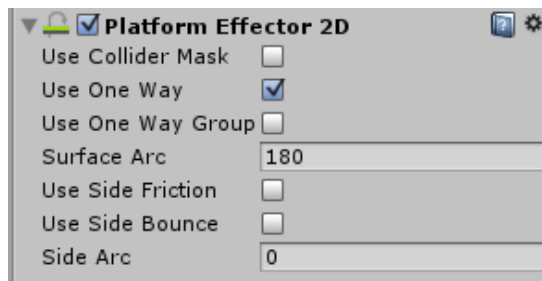


LUIZ MELO
Tiny Platformer - Forest Ass...
(not enough ratings)
FREE

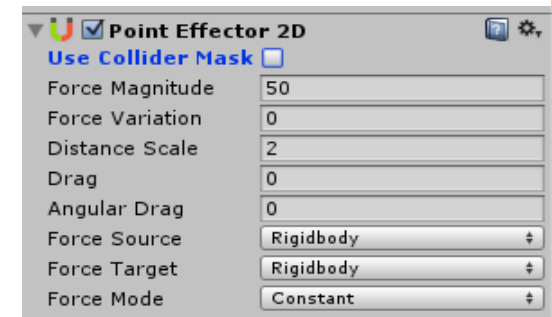


EFEKTORY – PRZYKŁAD ZASTOSOWANIA, C.D.

- Efektor platformy (przenikanie w jedną stronę) oraz efektor powierzchni (taśma); efekторы muszą być powiązane z odpowiednimi zderzaczami (used by effector)



- Zdefiniowanie efektora punktowego dla pustego obiektu wraz ze zderzaczem kołowym (dodatnia wartość Force Magnitude oznacza odpychanie obiektów od punktu)



EFEKTORY – PRZYKŁAD ZASTOSOWANIA, C.D.

- Skrypt tworzy n monet w losowych odległościach od obiektu. Obiekt ze skryptem zapisany jako prefabrykat

```
public class CoinSprayer : MonoBehaviour {
    public int numCoins = 10;
    public GameObject coinPrefab;
    public float offSetRange = 1.5f;

    void Start () {
        SpawnCoins();
    }

    void SpawnCoins() {
        for (int i = 0; i < numCoins; i++) {
            Vector2 spawnOffset = new Vector2 (Random.Range(-offSetRange, offSetRange), Random.Range(-offSetRange, offSetRange));
            Instantiate(coinPrefab, (Vector2)transform.position + spawnOffset, Quaternion.identity);
        }
    }
}
```

EFEKTORY – PRZYKŁAD ZASTOSOWANIA, C.D.

- Dowiązanie skryptu do postaci, który obsługuje zderzenie z wrogiem; W przypadku zderzenia uruchamiamy skrypt „rozrzucania monet”

```
public class DropCoins : MonoBehaviour {
    public GameObject coinSprayerPf;
    bool gotHit = false;

    void OnCollisionEnter2D (Collision2D col) {
        if (col.gameObject.CompareTag("Enemy") && !gotHit) {
            gotHit = true;
            Instantiate(coinSprayerPf, transform.position, Quaternion.identity);
        }
    }

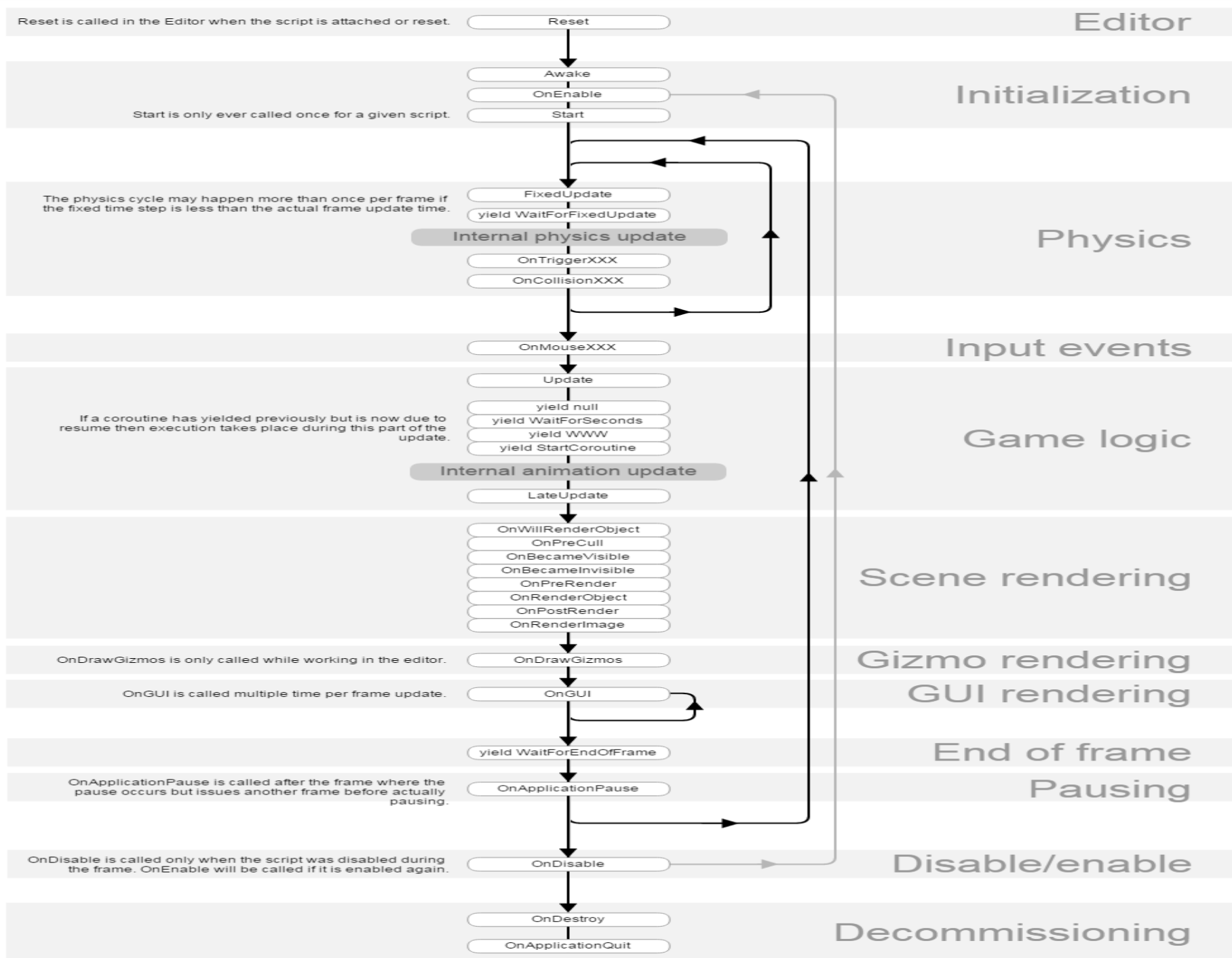
    void Start () {
    }
}
```



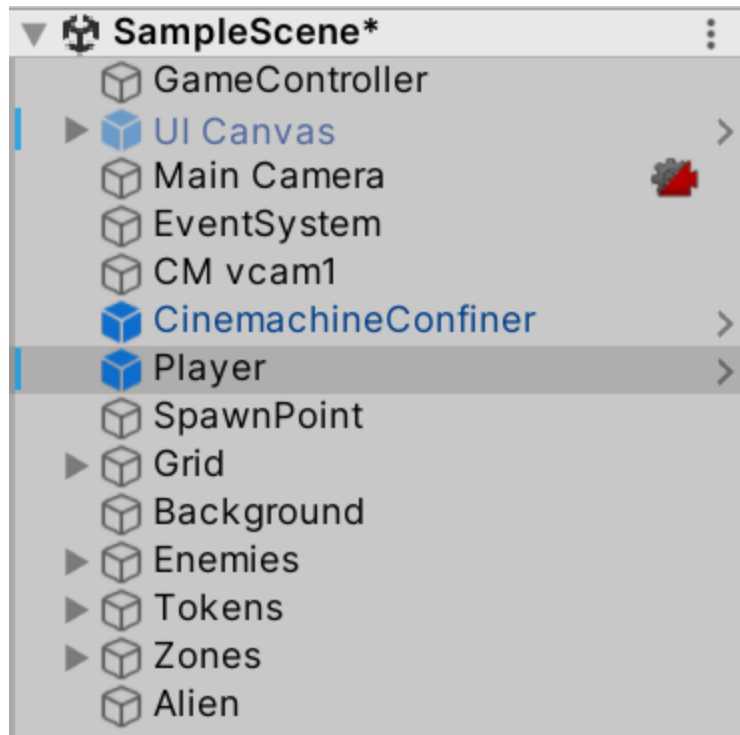
ZDARZENIA UPDATE

- FixedUpdate – zdarzenie wywoływane co pewien stały okres (częściej lub rzadziej niż Update); należy stosować do obliczeń zjawisk fizycznych. Można zmienić (Edit > Project Settings > Time > Fixed Timestep). Domyślnie ustawione na 0.02 (50 wywołań na s)
- Update – zdarzenie wywoływane raz na ramkę; tutaj wczytujemy wejście od użytkownika
- LastUpdate – zdarzenie wywoływane raz na ramkę, po ukończeniu obsługi Update; stosowane zwykle do pozycjonowania kamery, która porusza się za postacią



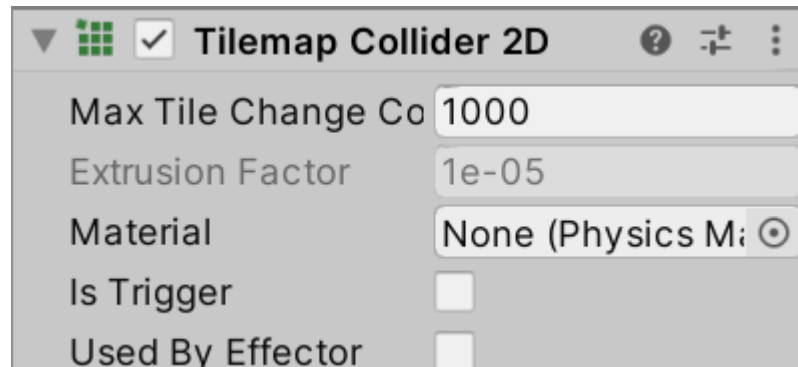
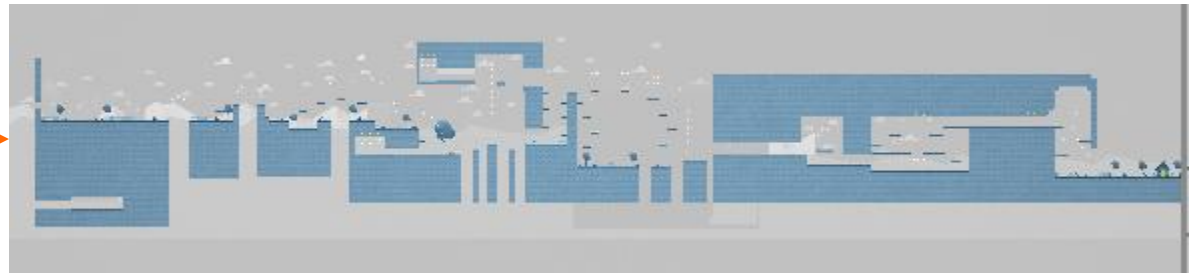
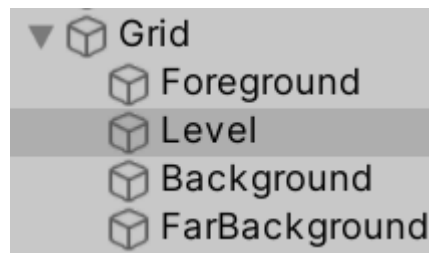


GRA 2D – 2D PLATFORMER MICROGAME (2.5)



GRA 2D – 2D PLATFORMER MICROGAME (2.5)

- Gra oparta o hierarchię obiektów zawierających TileMapy, rysowanych jedna nad drugą (patrz TileRenderer -> Layer)
- Obiekt Level ma komponent TileMap Collider 2D



GRA 2D – PLAYER

- Kinematyczna bryła sztywna (sterowana skryptem)
- Otoczona prostokątnym zderzaczem

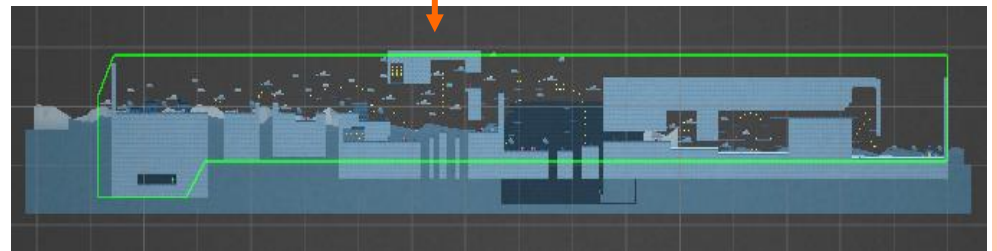
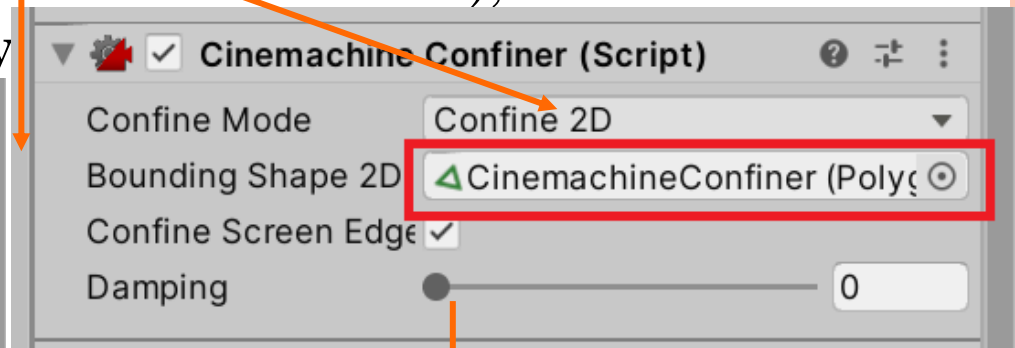
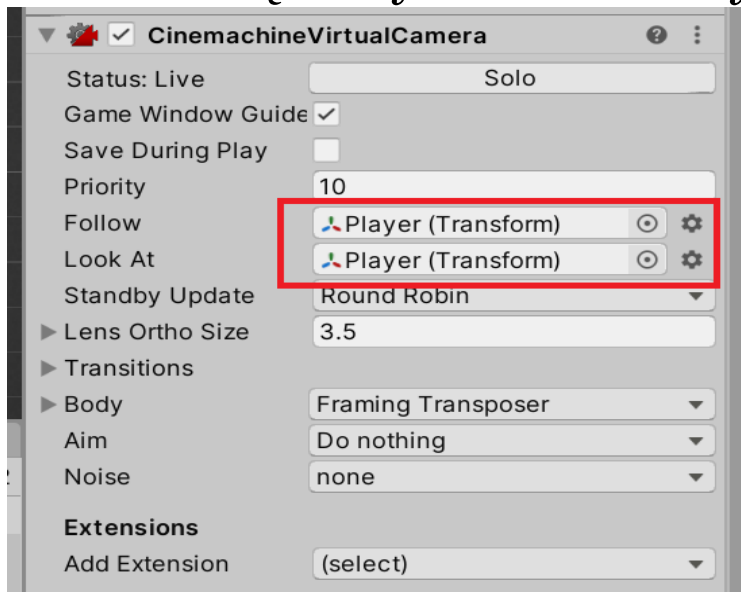
🔒 Unity Message | Odwołania: 2

```
protected override void Update()
{
    if (controlEnabled)
    {
        move.x = Input.GetAxis("Horizontal");
        if (jumpState == JumpState.Grounded && Input.GetButtonDown("Jump"))
            jumpState = JumpState.PrepareToJump;
        else if (Input.GetButtonUp("Jump"))
        {
            stopJump = true;
            Schedule<PlayerStopJump>().player = this;
        }
    }
    else
    {
        move.x = 0;
    }
    UpdateJumpState();
    base.Update();
}
```



GRA 2D – PLAYER

- Śledzenie gracza (kamera)
- System Cinemachine (wymaga zainstalowania)
 - Wirtualna kamera śledząca gracza
 - Nie wychodząca poza obszar
 - Sterowana mózgiem (CinemachineBrain), dołączonym do kamery



GRA 2D – PLAYER

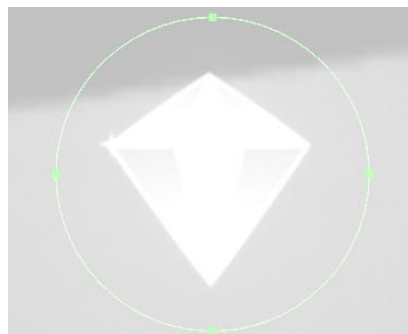
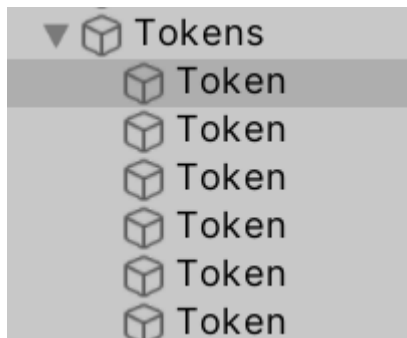
○ Upadek z platformy – Death zone

```
/// <summary>
/// DeathZone components mark a collider which will schedule a
/// PlayerEnteredDeathZone event when the player enters the trigger.
/// </summary>
Skrypt aparatu Unity (odwołania do zasobów: 4) | 1 odwołanie
public class DeathZone : MonoBehaviour
{
    Unity Message | Odwołania: 0
    void OnTriggerEnter2D(Collider2D collider)
    {
        var p = collider.gameObject.GetComponent<PlayerController>();
        if (p != null)
        {
            var ev = Schedule<PlayerEnteredDeathZone>();
            ev.deathzone = this;
        }
    }
}
```



GRA 2D – ZBIERANIE TOKENÓW

- Tokeny rozmieszczone na scenie z okrągłymi zderzaczami (trigger)

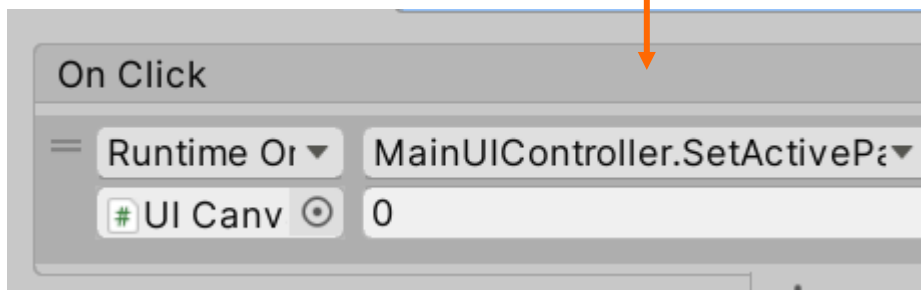
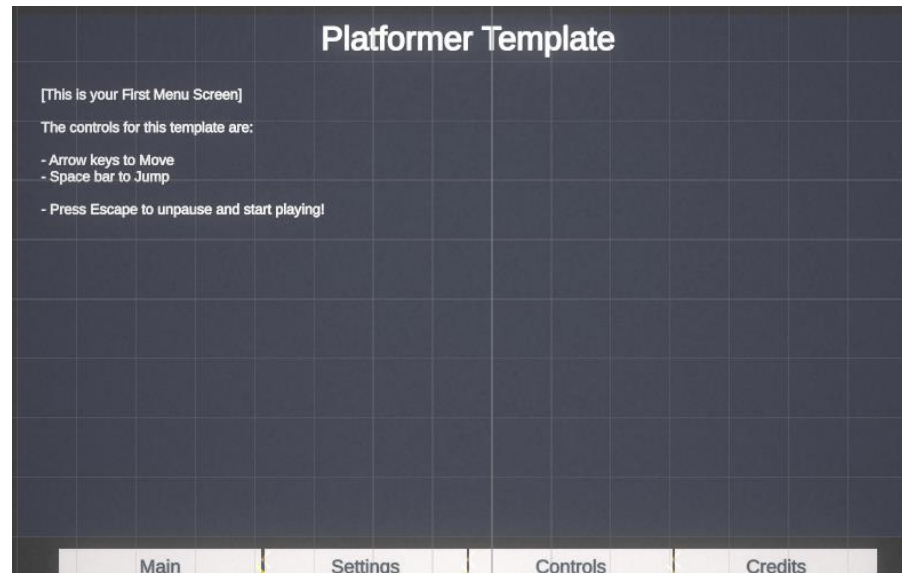
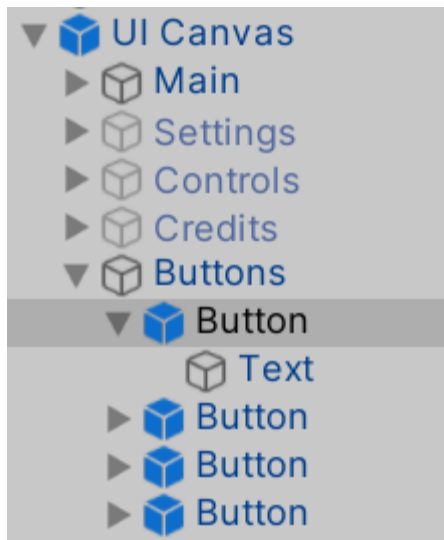


```
void OnTriggerEnter2D(Collider2D other)
{
    //only exectue OnPlayerEnter if the player collides with this token.
    var player = other.gameObject.GetComponent<PlayerController>();
    if (player != null) OnPlayerEnter(player);
}
```



GRA 2D – UI

- Kanwa z włączanymi/wyłączanymi elementami



GRA 2D (PLATFORMOWA)

- Przykładowe tutoriale (o różnym zakresie):
- <https://www.youtube.com/watch?v=on9nwbZngyw>
- <https://www.youtube.com/watch?v=A-GkNM8M5p8>
- <https://www.youtube.com/watch?v=soUg0HfpSlg>
- <https://www.youtube.com/watch?v=OR0e-1UBEOU>



PRZENOSZENIE DANYCH MIĘDZY SCENAMI

- Klasa z polami statycznymi (nie dziedziczająca z MonoBehaviour)
- Wykorzystanie metody DontDestroyOnLoad
- Klasa PlayerPrefs



METODA DONTDESTROYONLOAD

- Podczas ładowania nowego poziomu wszystkie obiekty na scenie są niszczone i tworzone są obiekty z nowego poziomu
- Metoda DontDestroyOnLoad pozwala „przenieść” obiekt (z danymi) między poziomami

```
public class GameController : MonoBehaviour {  
    public int health = 10;  
    public float experience = 20.0f;  
  
    void Start () {  
        DontDestroyOnLoad (gameObject);  
    }  
}
```



METODA DONTDESTROYONLOAD, C.D.

○ Wzorzec Singleton

```
public class GameController : MonoBehaviour {
    public int health = 10;
    public float experience = 20.0f;

    public static GameController controller;

    void Awake() {
        if (controller == null) {
            DontDestroyOnLoad (gameObject);
            controller = this;
        } else if (controller != this) {
            Debug.Log ("Destroy: " + this.health + " " + this.experience);
            Destroy (gameObject);
        }
    }

    void OnGUI(){
        GUI.Label (new Rect(10, 10, 100, 30), "Health: " + health);
        GUI.Label (new Rect (10, 40, 100, 30), "Experience:" + experience);
    }
}
```



KLASA PLAYERPREFS

- Rozwiązanie działające na wszystkich platformach
- Może być traktowana jako baza danych klucz-wartość
- Przechowuje dane między sesjami gry; sposób przechowywania zależy od systemu operacyjnego, np. dla Windows dane są przechowywane w rejestrze, w kluczu: HKCU\Software\[company name]\[product name] – company name i product name są ustawiane w ustawieniach projektu.



KLASA PLAYERPREFS

- Zapisywanie danych:

```
PlayerPrefs.SetString("PlayerName", "John");  
PlayerPrefs.SetInt("LastLevel", 2);  
PlayerPrefs.SetFloat("Brightness", 0.7f);  
PlayerPrefs.Save(); // teoretycznie można pominąć
```

- Odczytywanie danych:

```
string Name = PlayerPrefs.GetString("PlayerName");  
int LastLevel = PlayerPrefs.GetInt("LastLevel");  
float Brightness = PlayerPrefs.GetFloat("Brightness");
```

- Zarządzanie danymi:

- DeleteAll
- DeleteKey
- HasKey



PYTANIA KONTROLNE

- Jakie komponenty z pakietu Physics zostały omówione na wykładzie?
- Jakie cechy ma kinematyczna bryła sztywna?
- Czym się różni zderzacz, który jest wyzwalaczem (IsTrigger) od takiego, który nie jest?
- Czym jest efektor? Podaj 2 przykłady
- Jaka jest różnica między funkcjami FixedUpdate oraz Update?
- W jaki sposób można przenosić dane między scenami?

